devoteam

Artificial Intelligence Training
Kuehne + Nagel

Notebook Setup Guide

Workshop 2 · How LLMs Work · Python and Jupyter

Trainer: Rita Alves Ribeiro

2026-10-07 | v1.0

CONTENTS

1. What you will use 3

2. Install Python 3

3. Create the environment and install the libraries 3

4. Open the notebook 4

5. Check (3 minutes) 4

6. Common problems 4

Sources 5

Workshop 2 · How LLMs Work · 8 October 2026, 08:30-12:30

In the session, the hands-on exercises run in a Jupyter notebook that talks to the local model. This guide prepares an isolated Python environment for the training only, installs the libraries and checks that the notebook finds the model. Estimated time: 15 minutes.

Before you start: the local model must already be installed and working (Local Model Installation Guide).

1. What you will use

Component	Version	What for
Python	3.11 or later (3.12 recommended)	Running the notebook
Virtual environment (`venv`)	included with Python	Keeping the training libraries apart from the rest of the system
Jupyter or VS Code	Jupyter Notebook / JupyterLab, or VS Code with the Python and Jupyter extensions	Opening and running the notebook
Libraries	`requests`, `pandas`, `scikit-learn`, `matplotlib`	Talking to the model, tables, metrics and charts
Notebook folder	`K + N – W2 – Notebook` (sent by the trainer)	`kn_w2_how_llms_work.ipynb`, `helpers.py` and the `data/` folder

The notebook only talks to the local server (`localhost`): no request leaves the laptop.

Why a virtual environment? Installing libraries into the system Python fails on several platforms (the `externally-managed-environment` error on macOS with Homebrew and on recent Linux distributions), or mixes versions that clash with other tools. A virtual environment inside the notebook folder avoids both problems and can be deleted without leaving a trace.

2. Install Python

First check whether you already have a suitable version:

```
python3 --version        # macOS and Linux
py --version             # Windows
```

If it shows 3.11 or later, go to step 3.

Windows: install from https://www.python.org/downloads/ and, on the first installer screen, tick **Add python.exe to PATH**. Or, in PowerShell: `winget install Python.Python.3.12`.

macOS: install from https://www.python.org/downloads/, or with Homebrew: `brew install python@3.12`.

Linux:

```
# Ubuntu / Debian
sudo apt install python3 python3-venv python3-pip

# RHEL / Rocky / Alma 9 (the system python3 is 3.9, too old)
sudo dnf install python3.12
```

3. Create the environment and install the libraries

Open a terminal **inside the notebook folder** (`K + N – W2 – Notebook`).

macOS and Linux

```
python3 -m venv .venv          # on RHEL: python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install jupyter ipykernel requests pandas scikit-learn matplotlib
```

Windows (PowerShell)

```
py -3 -m venv .venv            # uses the newest Python 3 installed
.venv\Scripts\python -m pip install --upgrade pip
.venv\Scripts\python -m pip install jupyter ipykernel requests pandas scikit-learn matplotlib
```

On Windows, the commands call the environment's Python directly (`.venv\Scripts\python`): you do not need to activate it, and PowerShell's script blocking does not get in the way.

Finally, register the environment as a Jupyter *kernel*, with a name that is easy to find:

```
python -m ipykernel install --user --name kn-training --display-name "Python (kn-training)"

# Windows: .venv\Scripts\python -m ipykernel install --user --name kn-training --display-name "Python (kn-training)"
```

4. Open the notebook

Choose **one** option.

Option A: Jupyter in the browser

With the environment active, in the notebook folder:

```
jupyter notebook kn_w2_how_llms_work.ipynb

# Windows: .venv\Scripts\jupyter notebook kn_w2_how_llms_work.ipynb
```

A browser tab opens. Check in the top-right corner that the kernel is **Python (kn-training)**; if not, change it under *Kernel → Change kernel*.

Option B: VS Code

1. Install the **Python** and **Jupyter** extensions (by Microsoft).
2. *File → Open Folder…* and open the notebook folder.
3. Open `kn_w2_how_llms_work.ipynb`.
4. Click **Select Kernel** (top right) → *Python Environments* → choose `.venv` (or **Python (kn-training)** under *Jupyter Kernel*).

5. Check (3 minutes)

With Ollama running, or LM Studio with the server on and Ministral loaded:

1. **Install cell** (`%pip install ...`): run it with `Shift + Enter`. If it installs anything, restart the kernel (*Restart*).
2. **Setup cell** (section 0): it should print a line such as `Using ollama at http://localhost:11434 · model: ministral-3:3b` or `Using lmstudio at http://localhost:1234 · model: mistralai/ministral-3-3b`, followed by the list of installed models.
3. **Check cell:** it should answer `ready` (the first call takes longer, because it loads the model).

If all three steps work, you are ready. There is no need to run the rest of the notebook before the session.

- Python 3.11 or later
- `.venv` environment created and libraries installed
- Notebook open with the **Python (kn-training)** or `.venv` kernel
- The setup cell finds the runtime and the model
- The check cell answers `ready`

6. Common problems

Symptom	Likely cause	What to do
`ModuleNotFoundError: No module named 'pandas'` (or another library)	The notebook is using another kernel, not the environment's	Choose the **Python (kn-training)** or `.venv` kernel (step 4); restart the kernel
`error: externally-managed-environment`	`pip install` outside the virtual environment	Create and use the environment from step 3
`numpy.dtype size changed` or another binary error when importing pandas	Old libraries from another Python mixed in	Use the new environment from step 3 and its kernel
`RuntimeError: No usable local model found`	Ollama is not running, the LM Studio server is off, or there is no model	Open Ollama, or start the LM Studio server and load Ministral; run the setup cell again. The message says what it found
`cannot import name ... from 'helpers'`	An old `helpers.py`, or a kernel holding an old version in memory	Check that `helpers.py` is in the same folder as the notebook; restart the kernel
`ServerError: 400 ...` with long prompts	The prompt is larger than the context window loaded in LM Studio	Reload the model with *Context Length* 8192
`.venv\Scripts\activate` blocked in PowerShell	Script execution policy	No need to activate: use `.venv\Scripts\python` as in step 3

Symptom	Likely cause	What to do
`pip install` fails with a timeout or certificate error	Company proxy	`pip install --proxy http://<proxy>:<port> ...` with the details from IT, or install outside the company network
The **Python (kn-training)** kernel does not show in VS Code	Out-of-date kernel list	Close and reopen VS Code, or pick the `.venv` interpreter under *Python Environments*

Sources

Checked on 2026-10-07.

- Python, venv: https://docs.python.org/3/library/venv.html
- VS Code, Jupyter Notebooks: https://code.visualstudio.com/docs/datascience/jupyter-notebooks
- Jupyter, install: https://jupyter.org/install